

Busca Digital

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- No problema geral de **busca**, admite-se que existe conjunto de chaves $S = \{s_1, \dots, s_n\}$ e um valor x que se deseja localizar
- Métodos vistos consistiam em estruturar S de alguma forma conveniente e, através de comparações de x com chaves s_i de S , tentar localizar x em S
 - Chaves indivisíveis!!

Introdução (2)

- Cada chave armazenada em memória do computador (primária ou secundária) de forma eficiente
 - **Tamanho** de cada chave não excedia o da palavra
- Estruturas vistas possuíam **restrição** que chaves fossem todas de mesmo **tamanho**
 - Ou o **tamanho** da maior chave seria tomado como padrão
- Entretanto, nem sempre essas premissas se verificam

Introdução (3)

- Suponha armazenar um texto literário em computador:
 - Deseja-se buscar frases nesse texto
 - Conjunto S das chaves corresponderia às frases armazenadas
 - s_i : uma frase passível de ser buscada
 - Chaves podem assumir tamanho bastante diverso
 - Não seria possível armazenar cada chave em uma palavra do computador

Busca Digital

- Na **busca digital**, a chave é **divisível**
- Cada chave é constituída de um conjunto de caracteres (ou dígitos) de um **alfabeto**
- Em vez de se comparar a chave procurada com as chaves do conjunto armazenado, a comparação é efetuada, individualmente, entre os **dígitos** que compõe as chaves, dígito a dígito

Busca de Chave

- Seja $S = \{s_1, \dots, s_n\}$ um **conjunto** de n **chaves** em que cada s_i é formada por uma **sequência** de elementos d_j denominados **dígitos**
- Supõe-se que existe, em S , o total de m **dígitos** distintos, cujo conjunto forma o **alfabeto** de S
- Os dígitos do alfabeto admitem a ordenação $d_1 < \dots < d_m$
- Os p primeiros dígitos de uma chave formam o **prefixo de tamanho p** da chave

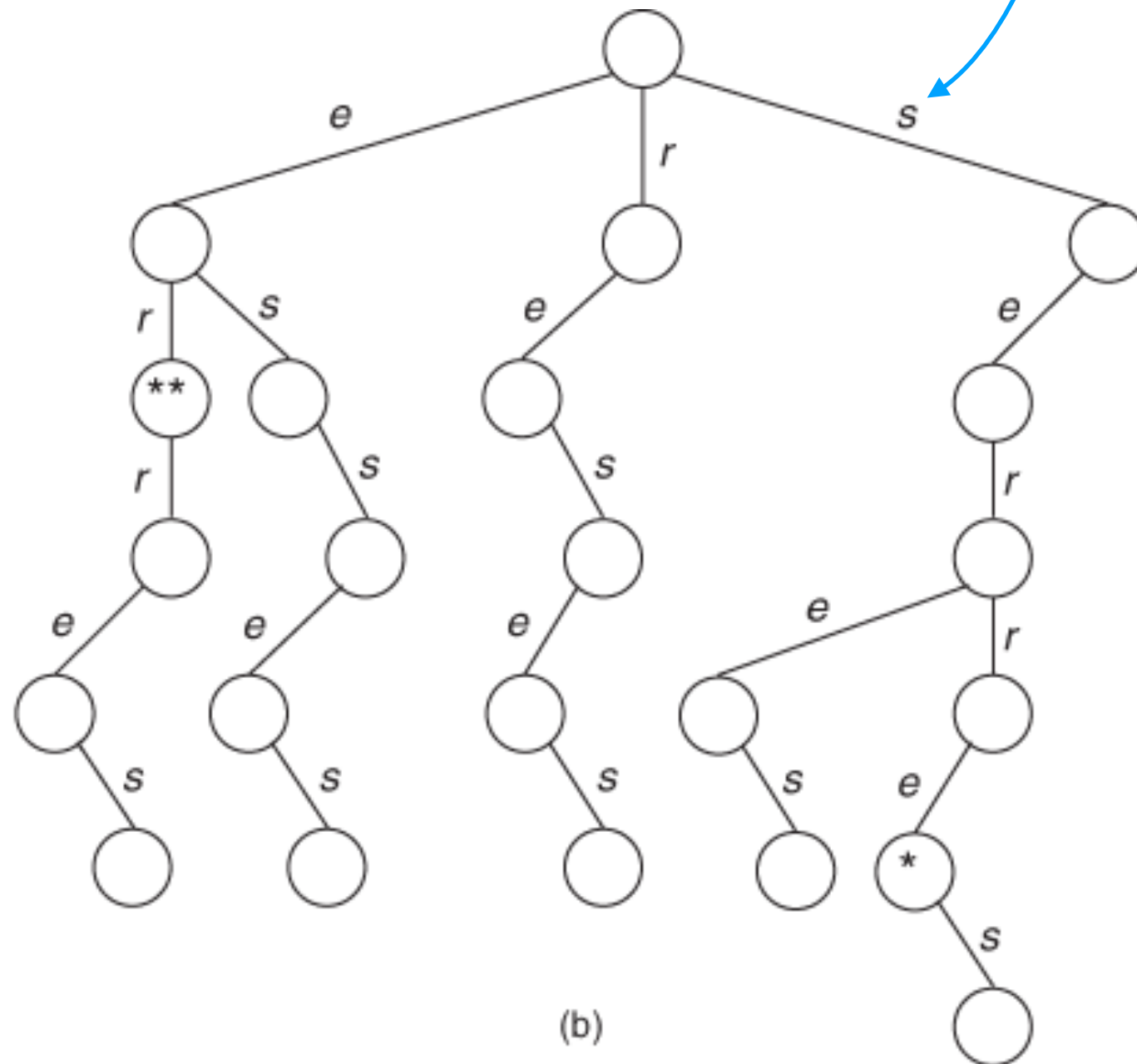
Árvore Digital (*Trie*)

- Uma **árvore digital** para S é uma árvore m -ária T , não vazia, tal que:
 - (i) Se um nó v é o j -ésimo filho de seu pai, então v corresponde ao dígito d_j do alfabeto S , $1 \leq j \leq m$.
 - (ii) Para cada nó v , a sequência de dígitos definida pelo caminho desde a raiz de T até v corresponde a um prefixo de alguma chave de S
- Decorre dessa definição que a raiz de uma árvore digital sempre existe e não corresponde a qualquer dígito do alfabeto

Figura: Árvore Digital

O dígito correspondente a cada nó encontra-se representado na aresta entre o nó e seu pai

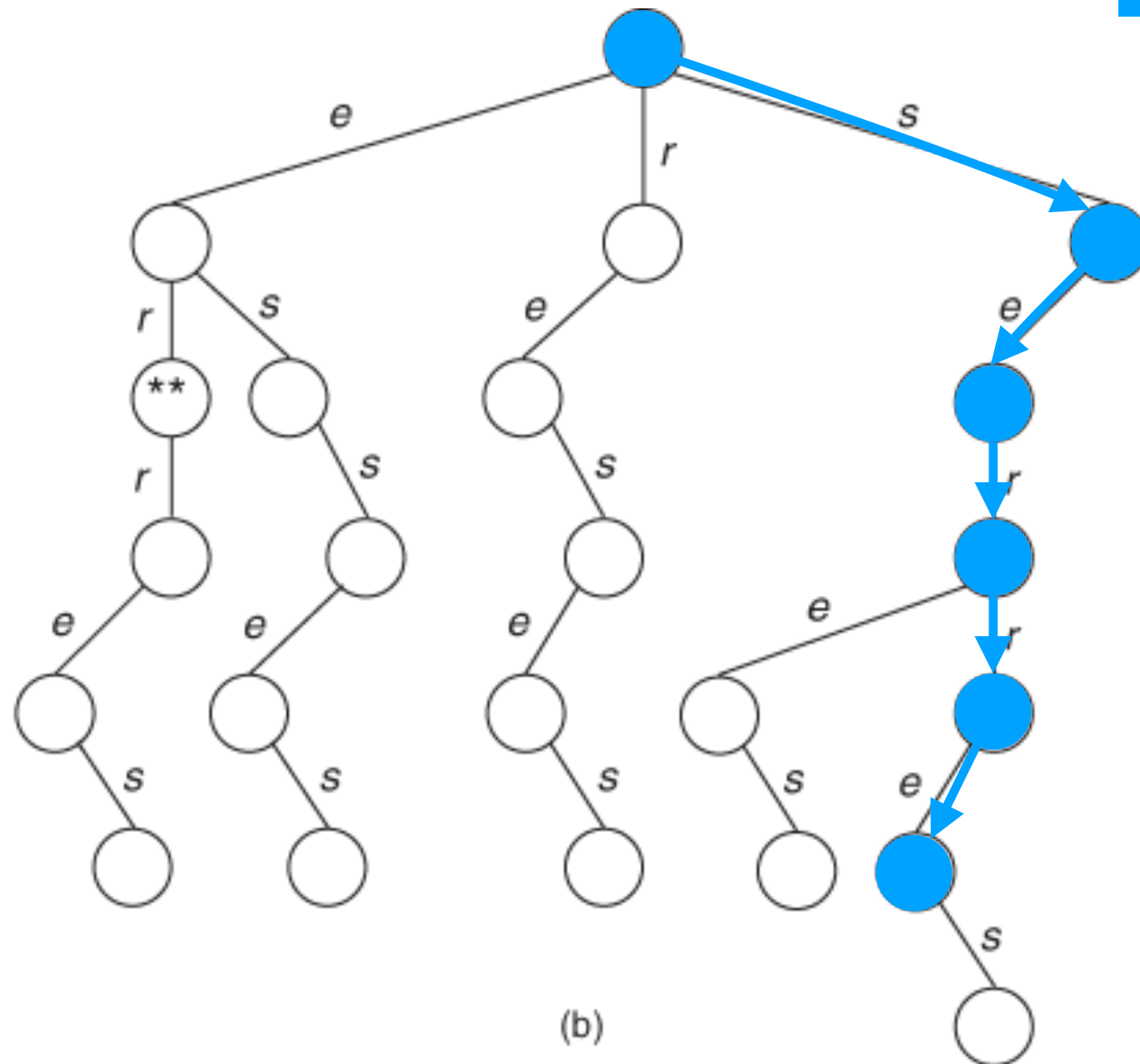
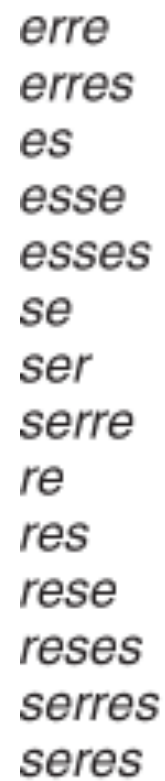
erre
erres
es
esse
esses
se
ser
serre
re
res
rese
reses
serres
seres



Árvore digital. O alfabeto é $\{e, r, s\}$. Portanto, $m = 3$ e a ordenação alfabética indica $e < r < s$. A figura ilustra um conjunto de chaves e uma árvore digital para essas chaves. É uma árvore ternária.

Figura: Busca em Árvore Digital (1)

$x = \textit{serre}$



Árvore de Ponteiros

- Nó com (*) corresponde à chave *serre*
- Em uma implementação da árvore digital, esse nó poderia conter um ponteiro para localizar a informação cuja chave é igual a *serre*
 - Nós que não correspondessem ao último dígito de uma **chave válida** conteriam um ponteiro nulo
- A árvore digital seria então uma **árvore de ponteiros**

Prefixos

- A busca localiza também **prefixos** de chaves válidas, que por si só não são chaves válidas
- Por exemplo, $x = er$, a busca terminaria com sucesso no nó assinalado **(**)**
 - Se v é **não terminal** de alguma chave do conjunto, então x é **inválida**

Algoritmo Busca Digital

```
procedimento buscadig( $x, pt, \ell, a$ )  
  se  $\ell < k$  então  
    seja  $j$  a posição de  $d(\ell + 1)$  na ordenação do alfabeto  
    se  $pt \uparrow . pont[j] \neq \lambda$  então  
       $pt := pt \uparrow . pont[j]; \ell := \ell + 1$   
      buscadig( $x, pt, \ell, a$ )  
  senão se  $pt \uparrow . info = terminal$  então  $a := 1$ 
```

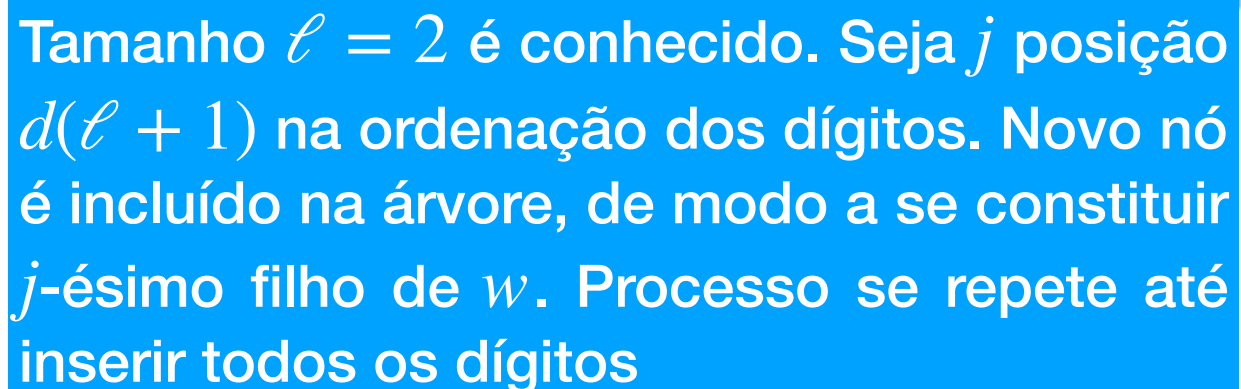
Inclusão em Árvore Digital

- Para incluir nova chave $x = d(1), \dots, d(k)$ numa árvore digital m -ária, utiliza-se o algoritmo de busca digital
- Se x foi localizado na árvore $\rightarrow$ inclusão inválida
- Senão
 - Busca determina o nó w da árvore, apontado por pt , tal que o caminho da raiz até w corresponde ao maior **prefixo** de chave que coincide com x
- Tamanho ℓ desse **prefixo** também é conhecido

Inclusão em Árvore Digital (Cont.)

- Seja j a posição de $d(\ell + 1)$ na ordenação de dígitos
- Então, um novo nó é incluído na árvore, de modo a se constituir o j -ésimo filho de w
- Processo se repete até que todos os dígitos da chave tenham sido esgotados

$x = reese$



Construção de Árvore Digital

- P/ **construir** árvore digital, basta utilizar algoritmo inclusão
 - Começa com uma árvore vazia (uma árvore contendo unicamente a raiz)
 - Inclui chaves uma a uma

Complexidade Busca

- Maioria dos casos práticos o valor de m é pequeno
- Complexidade $O(k)$ se bem projetado, onde k é o tamanho da chave

Complexidade Inclusão

- Decompõe-se o tamanho k da chave x , a ser incluída, em duas partes k_1 e k_2 ($k_1 + k_2 = k$)
 - k_1 : tamanho do maior prefixo comum a x e alguma chave
 - k_2 : número de nós a serem incluídos na árvore resultante da inclusão da chave x
- Cada nó incluído requer $O(m)$ operações
- Algoritmo produz k_1 iterações do procedimento **busca**
- Logo, a complexidade total é $O(k_1 + k_2m)$

Análise Árvore Digital

- Na **busca digital**, complexidade depende tão somente dos tamanhos da chave e do alfabeto
- Essa característica torna o método de busca uma base importante para o tratamento de grandes arquivos

Tamanhos Chaves & Prefixo

- Uma vantagem é que chaves podem possuir **tamanho arbitrário** e **variável**
- Uma propriedade interessante dessa busca é que, ao localizar um certo **prefixo** x , terão sido localizadas todas as chaves do arquivo cujo **prefixo** é igual a x

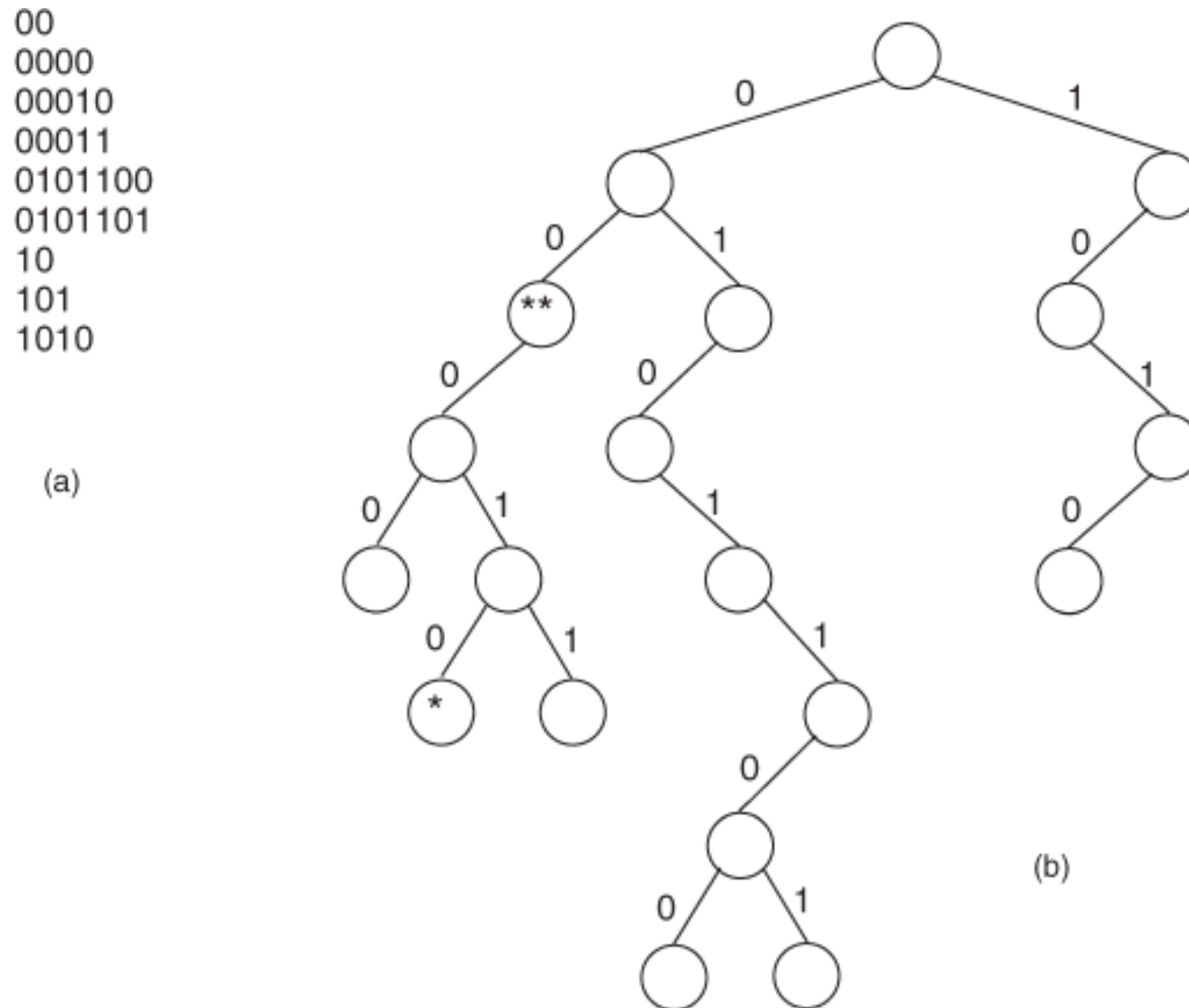
Excessiva Quantidade de Memória

- **Árvore digital** é, por vezes, **inviável**:
 - Excessiva quantidade de memória
 - Se houver n chaves de tamanho (máximo) t com prefixos distintos, então uma árvore digital m -ária necessitaria de espaço $O(nmt)$
- É tão mais eficiente quanto maior for a quantidade de chaves com **prefixos** comuns
- Muitos zigue-zagues quase sempre é **ineficiente**

Árvore Digital Binária

- Uma **árvore digital binária** é, simplesmente, o caso binário da árvore digital ($m = 2$)
 - Alfabeto = $\{0,1\}$
 - Cada chave é uma sequência binária
 - A seleção do filho esquerdo de um nó é interpretado como o dígito 0, e o direito como 1
- **Vantagem:** número de ponteiros vazios é menor
- Formação de **indesejáveis** zigue-zagues ainda permanece

Figura: Árvore Digital Binária



Árvore digital binária. O nó assinalado (*) na figura corresponde à chave 00010.

Árvore Digital Binária (Cont.)

- Existem chaves binárias que são **prefixos** de outras da coleção
 - Por exemplo, a chave 00 é prefixo da chave 00010
 - Caminho da raiz até o nó (**) corresponde à chave 00 é parte daquele até o nó (*), que, por sua vez, corresponde a 00010
 - Frequentemente, para melhor manipular a estrutura deseja-se que tal situação não aconteça!!

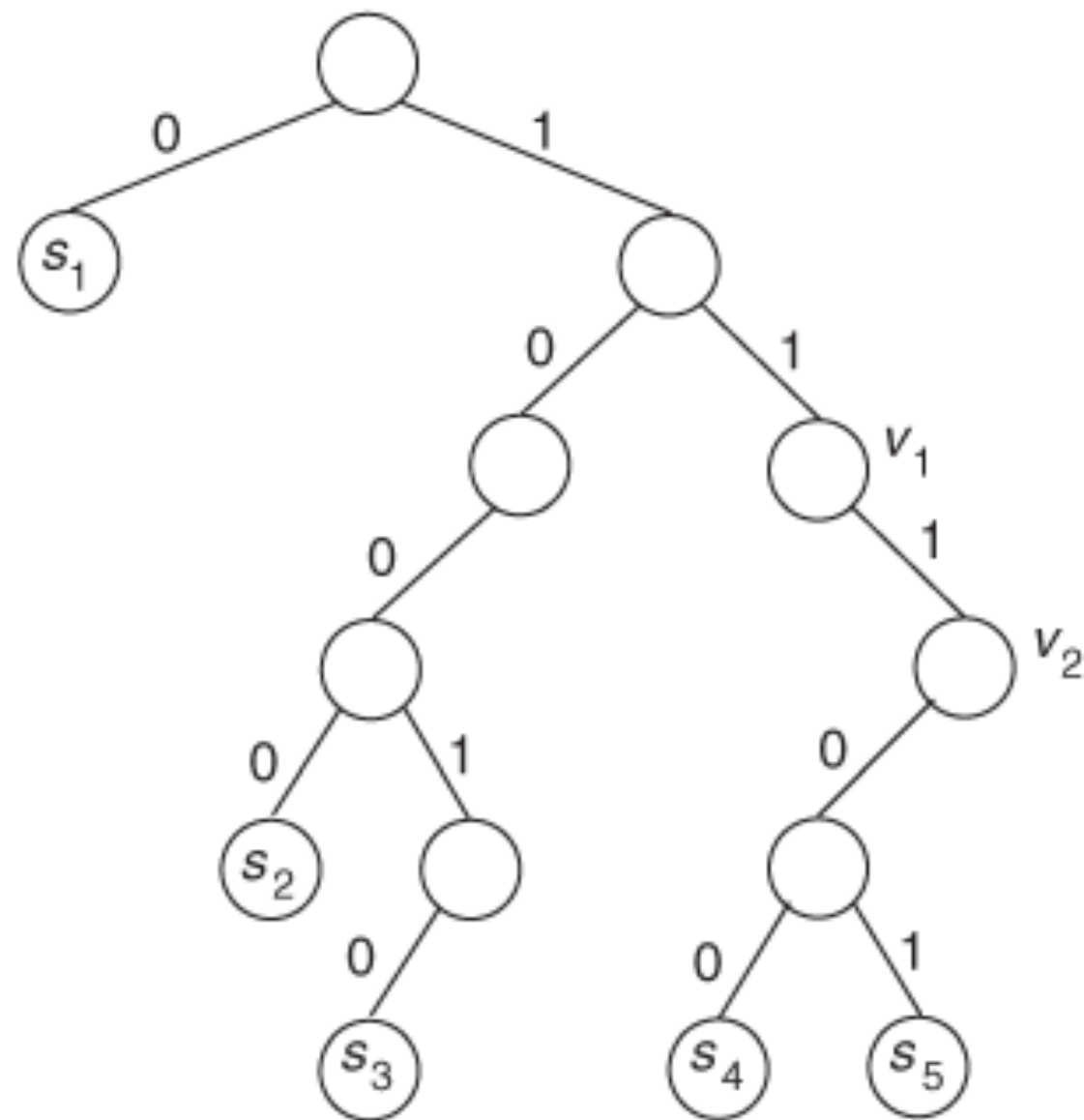
Árvore Binária de Prefixo

- Define-se, então, uma **árvore binária de prefixo**
 - Uma árvore digital binária tal que nenhum código seja prefixo de outro!!

Figura: Árvore Digital Binária de Prefixo

s_1 - 0
 s_2 - 1000
 s_3 - 10010
 s_4 - 11100
 s_5 - 11101

(a)



(b)

Cada chave é unicamente representada por uma folha e a codificação binária dessa chave corresponde ao caminho da raiz até essa folha.

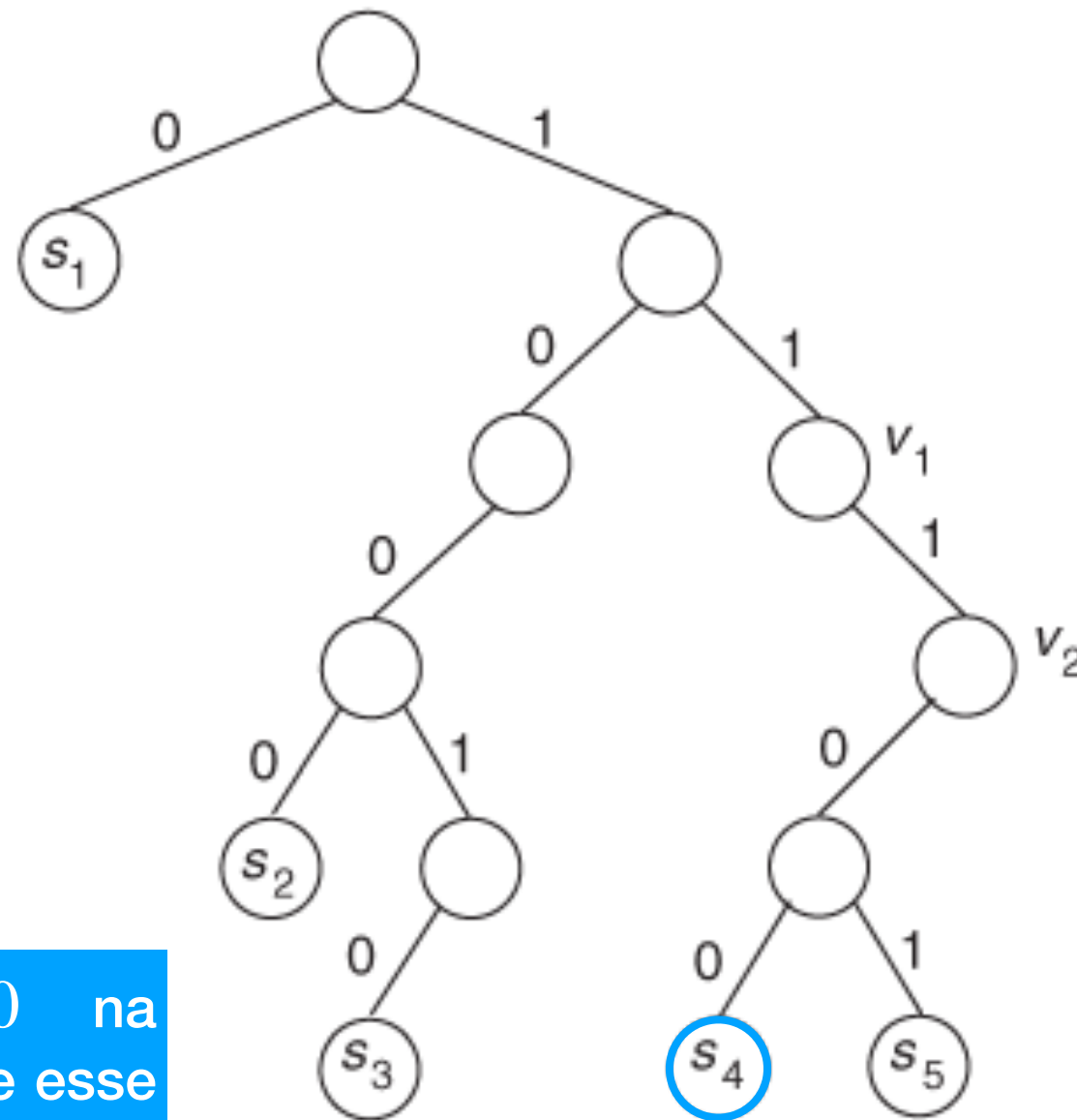
Árvore Patricia

- Construída a partir da árvore binária de prefixo
- Seja um conjunto de chaves $\{s_1, \dots, s_n\}$ de valores binários tais que nenhuma chave seja prefixo de outra
- Chaves válidas se encontram nas folhas da árvore

Figura: Busca em Árvore Digital de Prefixo

s_1 - 0
 s_2 - 1000
 s_3 - 10010
 s_4 - 11100
 s_5 - 11101

(a)



(b)

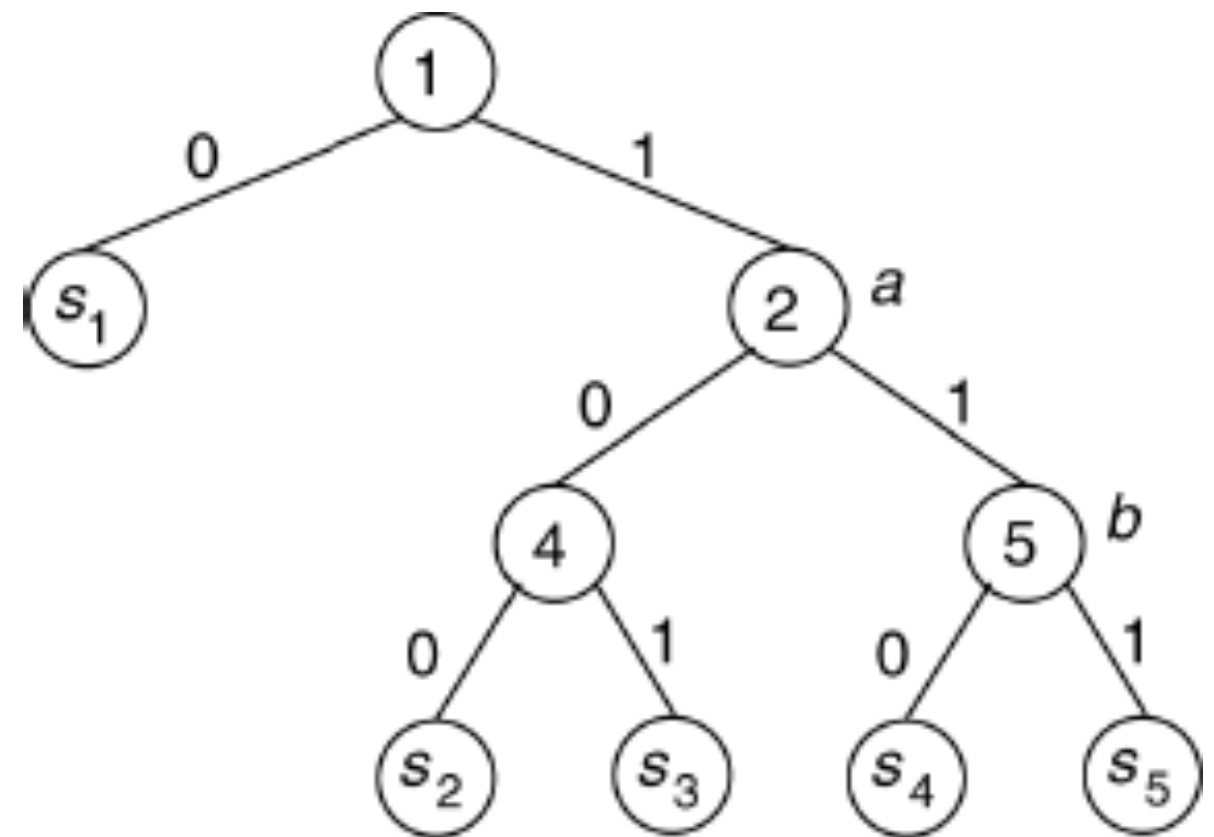
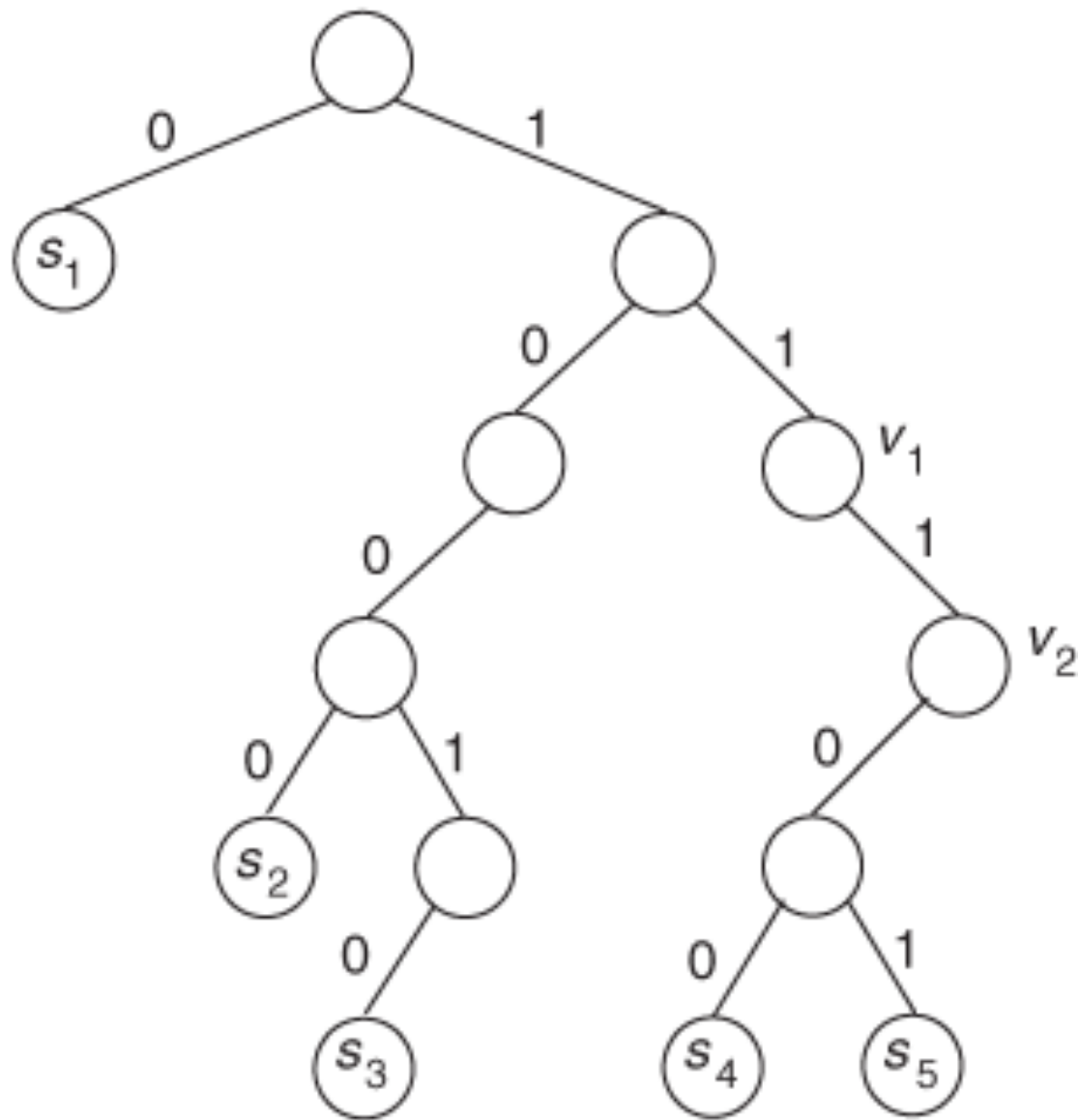
Algoritmo percorrerá caminho 11100 na árvore, iniciando pela raiz. Observe que esse caminho contém o zigue-zague formado pelos vértices v_1 , v_2 assinalados.

Buscar a chave $x = s_4$

Construção Árvore Patricia

- Seja H uma árvore binária de prefixo com mais de um nó. Realiza-se seguinte operação p/ cada zigue-zague $v_1, \dots, v_k$ de H :
 - Se v_k é uma **folha**, relativa à chave s_i
 - $\Rightarrow$ Compactar $v_1, \dots, v_k$ em v_1
 - $\Rightarrow$ Definir $r(v_1) = s_i$
 - Caso contrário, v_k possui o filho w
 - $\Rightarrow$ Compactar $v_1, \dots, v_k, w$ em v_1
 - $\Rightarrow$ Definir $r(v_1) = \text{nivel}_H(v_1) + k$
 - Se algum vértice v de T permaneceu sem rótulo:
 - $\Rightarrow$ Definir $r(v) = \text{nivel}_H(v)$
 - Se H possui um único nó:
 - $\Rightarrow$ Então T (**árvore patricia**) é vazia

Figura: Árvore Patricia Obtida

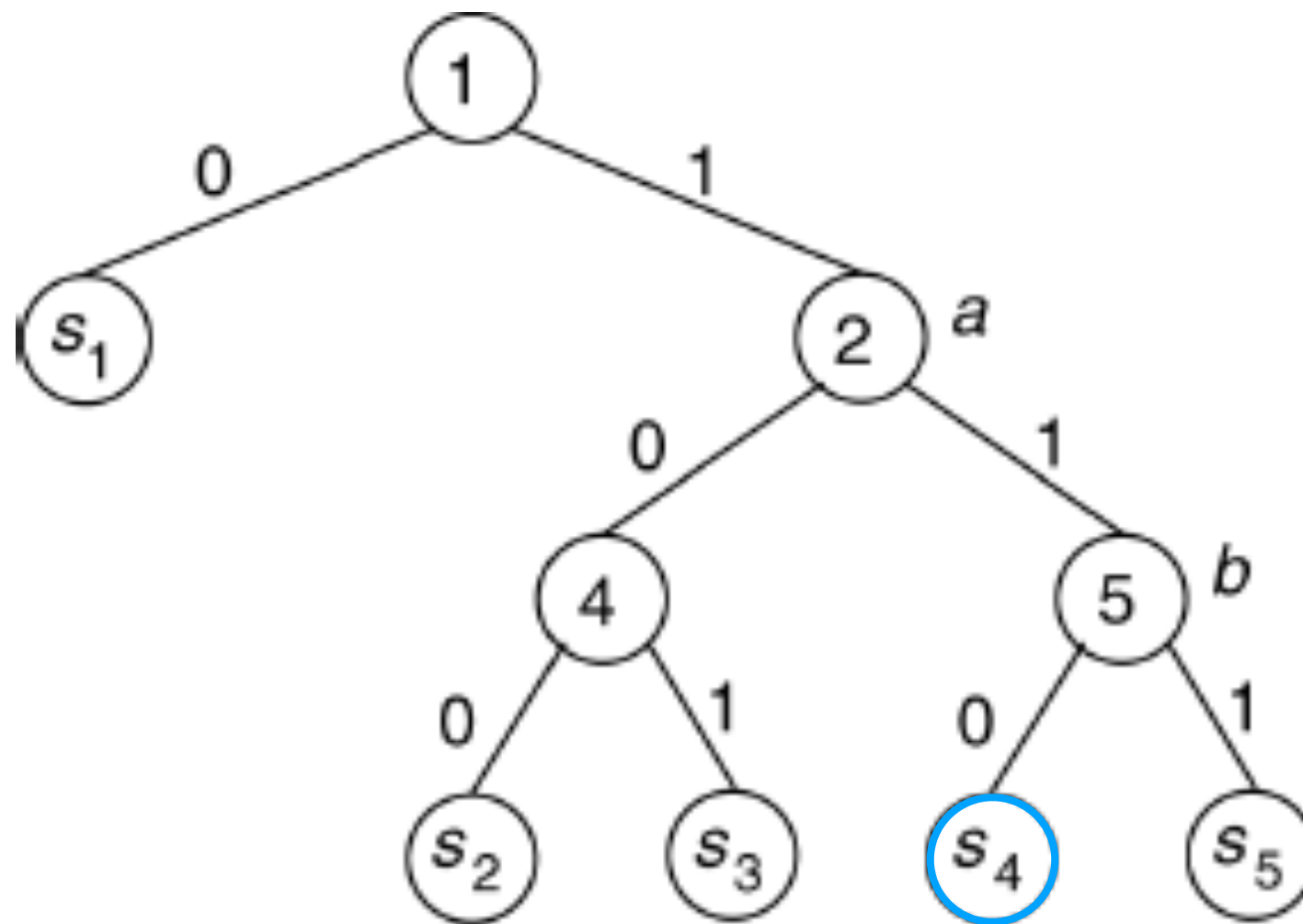


(À esquerda) Árvore de prefixo. (À direita) Árvore Patricia obtida.

Algoritmo Busca em Árvore Patricia

```
procedimento buscapat(x, pt, a)
  se pt->esq =  $\lambda$  então
    a := 1
  senão
    se k < pt->r então
      a := 2
    senão se d[pt->r] = 0 então
      pt := pt->esq
      buscapat(x, pt, a)
    senão
      pt := pt->dir
      buscapat(x, pt, a)
```


Figura: Busca em Árvore Patricia



Buscar a chave 11100

Análise Busca

- Complexidade é igual à altura da árvore
 - Limitada pelo tamanho da maior chave
- Eliminação dos zigue-zagues pode diminuir o tempo de busca
- Algoritmo de busca apresentado é proporcional ao tamanho da maior chave, devido a comparação $x = r(w)$
- **Ganho** da árvore Patrícia em relação à árvore digital binária fica restrito à memória

Inclusão

- Considere, agora, o problema de incluir, numa árvore Patricia T , uma nova chave x composta de k dígitos binários $d(1), \dots, d(k)$
- ...

Referências

- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian.
Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora:
LTC. 2010